

BiteKing POS & Billing System — Presentation Speaker Notes

Course: CST8219 – C++ Programming

Team Members: Arshdeep Sahota, Alhan Mahfouz, Syed Razi

Instructor: Prof. Sanaa | **Section:** 301

Total Target Time: 12–15 Minutes

Slide 1: Title Slide (30 Seconds) — [Presenter: Arshdeep Sahota]

Talking Points:

- "Good morning/afternoon Professor Sanaa and classmates. Today, our team is excited to present **BiteKing**, a modern, cross-platform Point-of-Sale and Billing System designed specifically for restaurant operations."
- "My name is Arshdeep Sahota, and I am presenting alongside my project partners, Alhan Mahfouz and Syed Razi."
- "Our project was built using modern **C++17**, the **Qt 6 Framework** for native graphical interfaces, and **CMake** for cross-platform build automation."

Tablet Cue / Transition:

- *Tap next slide as you finish introducing the team.*

Slide 2: Problem Statement / Motivation (1 Minute) — [Presenter: Arshdeep Sahota]

Talking Points:

- "To understand why we built BiteKing, let's look at the operational friction traditional restaurants face during peak rush hours."
- "Manual paper ticket logging and separate cash register processing slow down table turnaround, leading to billing errors, miscalculated taxes, and missed promotional discount applications."
- "Waitstaff need instant feedback on table availability, cashiers need validated change calculations, and managers need immediate visibility into daily revenue."
- "Our solution, **BiteKing**, solves this by providing a unified desktop terminal that bridges front-of-house order taking directly with automated tax calculations, payment processing, and dynamic menu analytics."

Key Metrics to Highlight:

- Automated 13% Harmonized Sales Tax (HST) calculation.
- Zero-delay real-time table state updates.

Slide 3: System Overview & Architecture (1–2 Minutes) — [Presenter: Arshdeep Sahota]

Talking Points:

- "Architecturally, BiteKing follows a clean multi-layered model separating the Qt Presentation Layer, the Management Controller Layer, the Domain Model, and Persistence."
- "The user interface is organized into five modular operating tabs:"
 1. **Menu Explorer:** Allows staff to search dishes, filter categories, and dynamically update catalog items.
 2. **Order Manager:** Binds dining table numbers to active orders, offering spinner inputs for quantity adjustments.
 3. **Billing System:** Provides a real-time preview of subtotals, happy-hour discounts, and exact tax totals before finalizing.
 4. **Payment Processing:** Handles both Cash and Card modes, enforcing tendered cash validation and change calculations.
 5. **Real-Time Analytics:** Computes overall sales totals and highlights top-selling menu items dynamically.

Hand-Off Cue / Transition:

- "Now, I'll pass the floor to Alhan and Syed to walk us through our C++ software class design and memory architecture."

Slide 4: Software Design & UML Architecture (2–3 Minutes) — [Presenters: Alhan Mahfouz & Syed Razi]

Talking Points:

- **Alhan Mahfouz:** "Thank you, Arshdeep. Let's examine the core object-oriented software design shown in our UML Class Diagram."
 - **Base Hierarchy:** "At the foundation of our domain model is the abstract base class MenuItem. It provides essential interfaces for item IDs, names, and base prices, which are inherited by concrete classes FoodItem (adding cuisine type, prep time, spiciness) and BeverageItem (adding volume and carbonation details)."
- **Syed Razi:** "Moving to object ownership and memory management:"
 - **Exclusive Ownership:** DiningTable manages CustomerOrder via std::unique_ptr, controlled by openOrder() and closeOrder(). When

RestaurantManager::completeTableOrder() is invoked upon checkout, the unique_ptr releases memory automatically.

- **Shared Memory:** "The RestaurantManager controller retains all catalog items inside std::vector<std::shared_ptr<MenuItem>>. Shared pointers are essential here because menu items are simultaneously referenced by active table order lines and billing previews."

Instructor Focus Point:

- Explicitly point out the distinction between std::unique_ptr (table order lifecycle) and std::shared_ptr (menu catalog items).

Slide 5: Core C++ Object-Oriented Principles (1–2 Minutes) — [Presenter: Alhan Mahfouz]

Talking Points:

- "BiteKing thoroughly applies the core pillars of Object-Oriented Programming required in CST8219:"
- **Encapsulation:** "All class member variables are kept private or protected. Public mutator methods enforce data integrity guardrails—preventing negative pricing or empty string identifiers."
- **Inheritance & Abstraction:** "MenuItem defines a pure virtual method virtual void displayItem() const = 0, making MenuItem an abstract base class that cannot be instantiated on its own."
- **Runtime Polymorphism:** "MenuItem declares virtual double calculateFinalPrice(double taxRate) and virtual double calculateDiscountedPrice(double discount). When iterating through a heterogeneous collection of std::shared_ptr<MenuItem>, C++ dynamically dispatches the correct overridden pricing logic for food vs. beverage items at runtime."

Slide 6: Key C++ Features & Advanced Concepts (2 Minutes) — [Presenter: Syed Razi]

Talking Points:

- "Beyond basic OOP, we integrated advanced C++ concepts into the project backend:"
- **Templates & SFINAE (Substitution Failure Is Not An Error):** Managed by AnalyticsEngine<T>::computeDailySales() and the internal SFINAE helper getItemValue(). Compile-time substitution directs computeDailySales() to call Bill::getTotal() or Payment::getAmount() seamlessly without runtime overhead.
- **Operator Overloading & Friend Functions:** "We overloaded the binary comparison operator operator== on MenuItem to compare item IDs, and defined a friend function operator<< for clean standard output logging without violating private encapsulation boundaries."

- **STL Containers:** "We utilized `std::vector` for cache-friendly sequential order storage and `std::map` for fast $O(\log n)$ table lookup queries."

Slide 7: GUI Demonstration & Qt 6 Architecture (2–3 Minutes) — [Presenter: Arshdeep Sahota]

Talking Points:

- **Arshdeep Sahota:** "Now, moving to the user interface, BiteKing was engineered using **Qt 6 Widgets**."
- **Responsive Layout Packing:** "We combined `QHBoxLayout`, `QVBoxLayout`, and `QGridLayout` wrappers to ensure the application resizes smoothly across different desktop display resolutions."
- **Event-Driven Signal & Slot System:** Controlled by `MainWindow` slots such as `on_addToOrderClicked()`, `on_tableComboChanged()`, and `updateOrderTableUI()`. Interacting with UI controls triggers Qt signals mapped directly to these C++ slots.
- **Dynamic QSS Styling:** "We implemented custom Qt Style Sheets (QSS) supporting dynamic Light and Dark mode themes to maintain high visual contrast in dark restaurant environments."

Slide 8: Data Persistence & File I/O (1 Minute) — [Presenter: Syed Razi]

Talking Points:

- **Syed Razi:** "Data persistence in BiteKing is managed without external heavy databases using custom file serialization:"
- **Polymorphic Serialization (`menu.txt`):** Executed by `MenuFileIO::loadMenu()` and `MenuFileIO::saveMenu()`. During startup, `loadMenu()` parses line type tags (FOOD or BEV) to instantiate appropriate derived classes via factory logic.
- **Transaction Logging:** "Our `DataManager` logs completed transactions, bills, and payment records to disk, preserving sales history across application restarts."

Slide 9: Testing Matrix & Edge Cases (1 Minute) — [Presenter: Arshdeep Sahota]

Talking Points:

- **Arshdeep Sahota:** "To ensure system stability, we executed comprehensive testing workflows covering edge cases:"
- **Input Guardrails:** "Item creation validates text length (e.g., rejecting single-character names) and blocks duplicate item IDs."
- **Boundary Validation:** "Order quantities are strictly bounded between 1 and 100 using

QSpinBox controls."

- **Financial Logic Validation:** "The cash payment module calculates change automatically and explicitly blocks transactions where tendered cash is less than the net bill amount."

Slide 10: Technical Challenges & Solutions (1 Minute) — [Presenter: Arshdeep Sahota & Team]

Talking Points:

- "During development, our team tackled key technical challenges across GUI styling, control rendering, input validation, and data persistence."
- **Challenge 1 — Linux GTK Theme Collision:**
 - *Problem:* On Linux dark desktop environments, unstyled QLineEdit search boxes inherited dark system text on dark backgrounds when switching themes, making text invisible.
 - *Solution:* We explicitly styled QLineEdit, QSpinBox, and QTableWidgetItem elements inside our stylesheet rules in applyTheme(), overriding default OS styles.
- **Challenge 2 — QComboBox Arrow Disappearance:**
 - *Problem:* Applying custom border styling disabled standard OS dropdown drawing in combo boxes, hiding native arrow indicators.
 - *Solution:* We implemented pure CSS/QSS downward arrows using CSS border trick geometry directly inside custom stylesheet rules (QComboBox::down-arrow).
- **Challenge 3 — Input Validation for Menu Items:**
 - *Problem:* Menu items have strict requirements – ID format, name length/letters, price range, valid categories. Invalid data must not crash the system.
 - - **Solution:** Enforced inside mutator methods like MenuItem::setPrice(), MenuItem::setName(), and FoodItem::setPrepTime(). Setting invalid values causes setters to throw validation warnings or revert to safe default fallbacks.
- **Challenge 4 — [Team Challenge / Open Q&A Spot]:**
 - *Note:* Reserved space on the slide for an additional team challenge or file I/O edge case during presentation defense.

Slide 11: Project Conclusion & Key Achievements (30 Seconds) — [Presenter: Arshdeep Sahota]

Talking Points:

- "In conclusion, BiteKing successfully achieves all learning outcomes for CST8219:"
- **Technical Summary:** "We built a modern, responsive, and 100% leak-free C++ desktop POS application using smart pointers, abstract polymorphism, SFINAE templates, and Qt GUI integration."
- **Future Scope:** "In future iterations, we plan to migrate plain-text storage to an embedded

SQLite database and add TCP/IP networking for multi-terminal sync."

Slide 12: Questions & Discussion (Q&A) — [Presenters: All Team Members]

Talking Points:

- "Thank you Professor Sanaa and classmates for your time and attention! We are now happy to answer any technical questions regarding our class design, C++ memory management, or Qt implementation."



Potential Instructor Q&A Cheat Sheet

1. **Q: Why did you use `std::shared_ptr` for `MenuItem` instead of raw pointers or `std::unique_ptr`?** [Answered by Syed Razi / Alhan Mahfouz]
 - A: "Multiple objects in our system need access to the exact same menu item instance at the same time—such as the catalog registry in `RestaurantManager`, active `OrderLine` items in a table order, and the billing window. `std::shared_ptr` allows reference sharing with automatic memory deallocation when the last reference drops."
2. **Q: How does `MenuItem` prevent instantiation of generic menu items?** [Answered by Alhan Mahfouz]
 - A: "By declaring virtual `displayItem()` `const = 0`; as a pure virtual function, `MenuItem` becomes an abstract class. The C++ compiler blocks any direct instantiation of `MenuItem`."
3. **Q: How did you prevent memory leaks when working with Qt Widgets?** [Answered by Arshdeep Sahota]
 - A: "Qt utilizes a parent-child ownership tree (`QWidget*` parent). When a parent window or container is destroyed, Qt automatically deletes all child widgets attached to it. For non-Qt domain model classes, we relied entirely on smart pointers (`std::unique_ptr` and `std::shared_ptr`)."